分块 --算法竞赛专题解析（25）

原创 罗勇军 2020-09-08 19:35:39 2788 收藏 5 版权

分类专栏： 高级数据结构

本系列文章将于2021年整理出版。前驱教材：《算法竞赛入门到进阶》 清华大学出版社
网购：京东 当当 作者签名书：**点我**
有建议请加QQ 群：567554289

文章目录

1. 分块概念
2. 分块算法
3. 复杂度分析
4. 例题

1. 分块概念

回顾“区间”问题，前面给出了暴力法、树状数组、线段树等算法。给定一个保存n个数据的数列，做m次“区间修改”和“区间查询”，每次操作只涉及到部分区间。暴力法只是简单地从整体上做修改和查询，复杂度O(mn)，很低效。树状数组和线段树都用到了二分的思想，以O(logn)的复杂度组织数据结构，每次只处理涉及到的区间，从而实现了O(mlogn)的高效的复杂度。

虽然暴力法只能解决小规模的问题，但是它的代码非常简单。

有一种代码比树状数组、线段树简单，效率比暴力法高的算法，称为“**分块**”，它能以$O(m\sqrt{n})$的复杂度解决“区间修改+区间查询”问题。简单地说，分块是用线段树的“分区”思想改良的暴力法；它把数列分成很多“块”，对涉及到的块做整体性的维护操作（类似于线段树的lazy-tag），而不是像普通暴力法那样处理整个数列，从而提高了效率。

用一个长度为n的数组来存储n个数据，把它分为t块，每块长度为n/t。下图(1)是一个有10个元素的数组，共分成4块，前3块每块3个元素，最后一块1个元素。

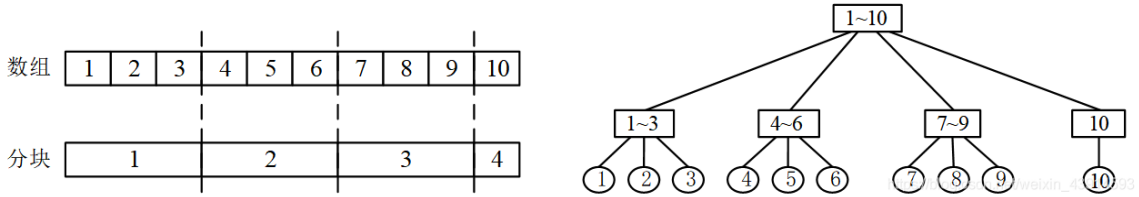


图1 (1)分块　　(2)与线段树的结构对比

对比块状数组与线段树，线段树是一棵高度为logn的树，块状数组可以看成一棵高度为3的树，见图(2)。从图(2)可知，在线段树上做一次操作是$O(logn)$的，因为它有logn层；分块是$O(n/t)$的，因为它把数据分成了t块，处理一块的时间是n/t的。下面介绍分块算法，并详细说明复杂度。

2. 分块算法

块操作的基本要素有：

（1）块的大小。用block表示。

（2）块的数量。用t表示。

（3）块的左右边界。定义数组st[]、ed[]，用st[i]、ed[i]表示块i的第一个和最后一个元素的位置。st[1] = 1，ed[1] = block；st[2] = block+1，ed[2] = 2×block；…；st[i] = (i-1)*block+1，ed[i] = i*block；…

（4）每个元素所属的块。定义pos[]，pos[i]表示第i个元素所在的块，pos[i]=(i-1)/block + 1。

具体内容见下面的代码。其中每块的大小block的值等于$\sqrt{n}$取整，后面的“复杂度分析”会说明原因。如果$\sqrt{n}$的结果不是整数，那么最后要加上一小块，代码中重要的内容是处理这个问题。

```
int block = sqrt(n);          //块的大小：每块有block个元素。
int t = n/block;              //块的数量：共分为t块
if(n % block) t++;            //sqrt(n)的结果不是整数，最后加一小块
for(int i=1; i<=t; i++){      //遍历块
    st[i] = (i-1)*block+1;
    ed[i] = i*block;
}
ed[t] = n;                    //sqrt(n)的结果不是整数，最后一块较小
for(int i=1; i<=n; i++)       //遍历所有元素的位置
    pos[i]=(i-1)/block + 1;
```

用分块解决区间问题很方便，下面以“区间修改+区间查询”（洛谷P3372）为例。

首先定义区间有关的辅助数组：

（1）定义数组a[]存储数据，共n个元素，读取初值，存储在a[1]、a[2]、…、a[n]中。

（2）定义sum[]，sum[i]为第i块的区间和，并预处理出初值。

```
for(int i=1; i<=t; i++)                 //遍历所有的块
    for(int j=st[i]; j<=ed[i];j++)      //遍历块i内的所有元素
        sum[i] += a[j];
```

（3）定义add[]，add[i]为第i块的增量标记，初始值为0。

然后对数列a[]做“区间修改+区间查询”操作：

（1）区间修改：将区间[L, R]内每个数加上d。

情况1，[L, R]在某个i块之内，即[L, R]是一个“碎片”。把a[L]、a[L+1]、…、a[R]逐个加上d，更新sum[i] = sum[i] + d*(R - L + 1)。计算次数约为n/t。

情况2，[L, R]跨越了多个块。在被[L, R]完全包含的那些整块内（设有k个块），更新add[i] = add[i] + d。对于不能完全包含的那些碎片（它们在k个整块的两头），按情况1处理。情况2的计算次数约为n/t + k，1 ≤ k ≤ t。

总结两种情况，处理碎块时，只更新sum[i]，不更新add[i]；处理整块时，只更新add[i]，不更新sum[i]。

```
void change(int L,int R,int d){
    int p = pos[L], q = pos[R];
    if(p==q){                       //情况1，计算次数是n/t
       for(int i=L;i<=R;i++)  a[i]+=d;
       sum[p]+=d*(R-L+1);
    }
    else{                           //情况2
       for(int i=p+1;i<=q-1;i++)    add[i]+=d; //整块，有m=(q-1)-(p+1)+1个。计算m次
       for(int i=L;i<=ed[p];i++)    a[i]+=d;   //整块前面的碎片。计算n/t次
       sum[p]+=d*(ed[p]-L+1);
       for( int i=st[q];i<=R;i++)   a[i]+=d;   //整块后面的碎片。计算n/t次
       sum[q]+=d*(R-st[q]+1);
    }
 }
```

（2）区间查询：输出区间[L, R]内每个数的和。

情况1，[L, R]在某个i块之内。暴力加每个数，最后加上add[i]，答案是ans = a[L] + a[L+1] + … + a[R] + (R - L + 1)*add[i]。计算次数约为n/t。

情况2，[L, R]跨越了多个块。在被[L, R]完全包含的那些块内（设有k个块），ans += sum[i] + add[i]*len[i]，其中len[i]是第i段的长度，等于n/t。对于不能完全包含的那些碎片，按情况1处理，然后与ans累加。计算次数约为n/t + k，1 ≤ k ≤ t。

```
long long ask(int L,int R) {
    int p=pos[L],q=pos[R];
    long long ans=0;
    if(p==q){                    //情况1
       for(int i=L;i<=R;i++)     ans += a[i];
       ans+=add[p]*(R-L+1);
    }
    else{//情况2
       for(int i=p+1;i<=q-1;i++)  ans+=sum[i]+add[i]*(ed[i]-st[i]+1);//整块
       for(int i=L;i<=ed[p];i++)  ans += a[i];   //整块前面的碎片
       ans += add[p]*(ed[p]-L+1);
       for(int i=st[q];i<=R;i++)  ans += a[i];   //整块后面的碎片
       ans += add[q]*(R-st[q]+1);
    }
    return ans;
 }
```

分块算法的实现简单粗暴，没有复杂数据结构和复杂逻辑，很容易编码。

分块算法的思想，可以概况为“**整块打包维护，碎片逐个枚举**[1]”。

3. 复杂度分析

把数列分为t块，t取何值时有最佳效果？

观察一次操作的计算次数n/t和$n/t+k$，其中$1 \le k \le t$；当$t=\sqrt{n}$时，有较好的时间复杂度$O(\sqrt{n})$。m次操作的复杂度是$O(m\sqrt{n})$，适合求解$m=n=10^5$规模的问题，或$O(m\sqrt{n}) \approx 10^7$的问题。对复杂度的直观理解，请看图1。

空间复杂度：需要分配长度为 的数组st[]、ed[]、sum[]、add[]和长度为n的pos[]、a[]，约3*MAXN，比线段树的9*MAXN好得多。不过，分块只能解决m = n = 10^5规模的问题，而线段树是10^6规模的，应用场景不同，直接对比空间无意义。

4. 例题

有些题目用普通的线段树、树状数组求解很难编码，而用分块比较容易。

例题1：区间第k大问题。

教主的魔法 洛谷P2801
题目描述：有N个数，有两种操作，区间修改（加）、区间询问。
输入：第1行有两个整数n、m。第2行有n个正整数。第3行到第m + 2行，每行是一个操作，有两种操作：
（1）第一个字母是“M”，后面三个数字L、R、W，表示对闭区间[L, R]内每个数加上W。
（2）第一个字幕是A，后面三个数字L、R、C，询问闭区间[L, R]内有多少数字大于等于C。
输出：对每个“A”询问输出一行，包含一个整数，表示大于等于C的数有多少个。
数据范围：n ≤ 1,000,000，m ≤3000，1 ≤ W≤1000，1 ≤ C≤1,000,000,000

题解：

如果用复杂度O(mn)的算法，不能通过测试。

询问区间[L, R]有多少数字大于等于C，等同于问C是区间第几大，即“区间第k大”问题，标准解法是主席树，m次操作的复杂度是O(mlogn)。

本题的n较小，用“分块 + 二分”算法，复杂度满足要求，而且代码很容易写。容易想到以下分块操作方法：

（1）首先读取数列a[]，把它分为$\sqrt{n}$块。

（2）区间修改。每个块维护一个add标记，用于记录块内的增量W；更新时，区间内的整块更新add，不完整的碎片，暴力更新其中的每个数。

（3）区间查询。大于等于C的数有多少？如果直接暴力搜每个块，复杂度为O(n)，不能满足要求。如果块中的数是有序的，那么用二分来找大于C的数，复杂度为O(logn)。但是块内的数是无序的，需要先排序再用二分（可以直接用lower_bound()函数），复杂度O(nlogn + logn)，还不如直接暴力搜。如果能“**一次排序，多次使用**”，就高效了。

下面是改进后的算法。

（1）在区间操作前，对每个块的初始值排序，复杂度O(nlogn)。不过，排序会改变原来元素的位置，所以定义一个辅助数组b[]，它的初值是数列a[]的复制，排序操作在b[]上进行。也就是说，b[]的每个块内部都是有序的，对b[]的某个块统计前k个数，就是对a[]的对应块统计前k个数。

（2）区间修改。如果是整块，维护add标记，不用在b[]上对整块再排序，因为它仍然保持有序；如果是碎片，暴力修改a[]上对应位置的数，然后把碎片所在的整块复制到b[]上，对这个块重新排序。复杂度 = 整块维护 + 碎片排序 = O($\sqrt{n} + \sqrt{n}log(\sqrt{n})$)。

（3）区间查询。对整块，因为已经是有序的，直接在b[]的对应整块上二分查询；对碎块，暴力搜a[]上的碎块。复杂度 = 整块查询 + 碎片查询 = O($\sqrt{n}log(\sqrt{n}) + \sqrt{n}$)。

做m次区间操作，以上三者相加，总复杂度是$O(nlogn) + O(m(\sqrt{n} + \sqrt{n}log(\sqrt{n})) \approx O(m\sqrt{n}log(\sqrt{n}))$。勉强通过测试。

例题2：hdu 5057

Argestes and Sequence hdu 5057

Time Limit: 5000/2500 MS (Java/Others) Memory Limit: 32768/32768 K (Java/Others)

题目描述：给定一个序列，有n个非负整数a[1], a[2],…, a[n]。做“单点修改 + 区间查询”操作。

输入：第一行是整数T，表示测试用例数量。对每个测试，第一行包含两个数字n、m。第二行是n个非负整数，用空格分割，后面有m行，每行表示一个操作，有两种操作：

S X Y: 修改操作，把a[x]的值置为y，即a[x] = y；

Q L R D P: 查询操作，询问区间[L, R]内有多少个数的第D位是P。

输出：对每个Q询问，输出一行答案。

数据范围：1≤T≤50，1≤n, m≤100000，0≤a[i]≤2^{31}-1，1≤X≤n，0≤Y≤2^{31}-1，1≤L≤R≤n，1≤D≤10，0≤P≤9

题解：

首先试试分块，看复杂度是否符合要求。

用分块编码非常容易。把数组分为$\sqrt{n}$块，然后定义block[i][D][P]，表示第i块第D位是P 的总个数。

（1）初始化。读取数组a[]的初值，根据a[]计算出block[][][]的初值。复杂度O(n)。

（2）修改操作。单点修改a[x]，根据a[x]更新block[][][]。复杂度O(1)。

（3）查询操作。在碎片上，暴力计算[L, R]内的每个a[]。在整块上，累加所有整块的block[][][]即可。复杂度 = 整块的计算 + 碎片的计算 = O($\sqrt{n}$) + O($\sqrt{n}$) = O($\sqrt{n}$)。

总复杂度 = 初始化 + m个操作 = O(n) + O(m$\sqrt{n}$)，勉强通过测试。

此题也可以用树状数组，并且这是一道练习树状数组的好题。树状数组的基础功能是“单点修改 + 区间查询”，符合本题的要求。

一个数据最多有D = 10位，每位有P = 0~9这10个数，所以询问共有D*P = 10*10 = 100种情况。

如果所有的操作只涉及一种情况，用树状数组很容易编程。例如所有的a[i]都只有1位，这1位要么是0，要么是1，然后询问区间[L, R]内有多少个1。这是最基本的树状数组。

但是，如果100种情况都用树状数组来处理，需要定义的树状数组是int tree[10][10][100000]，需要40M空间，超内存。所以必须把tree减少一维，即int tree[10][100000]，此时需要用离线操作的技巧。

（1）先读取并保存所有的修改和查询操作。

（2）“用时间换空间”，分10次处理所有的操作，第1次处理第1位，第2次处理第2位，…等等。每次处理用int tree[10][100000]，分别处理0~9这10个数；这相当于使用了10个树状数组tree[10][100000]。记录查询操作的结果。

（3）按顺序输出查询的结果。

计算复杂度是多少？上面的步骤，等于做了10次O(mlogn)的树状数组，注意不是做了100次，请思考原因。树状数组的效率比分块高很多，不过编码的难度要高很多倍。

例题3：洛谷 P3203

弹飞绵羊 洛谷 P3203

题目描述：一条直线上摆着n个弹簧，每个弹簧有弹力系数k_i，当绵羊到第i个弹簧时，它会被弹到第$i+k_i$个位置，若不存在第$i+k_i$个弹簧，则绵羊被弹飞。

绵羊想知道当它从第i个弹簧起步时，被弹几次后会被弹飞。为了使游戏有趣，允许修改某个弹簧的弹力。弹力系数始终为正。

输入：第一行包含一个整数n，表示地上有n个装置，编号0~n-1。接下来有n个正整数，依次为n个弹簧的初始弹力系数。第三行有一个正整数m，表示操作次数。接下来m行每行至少有两个数i，j。

若i=1，你要输出从j出发被弹几次后弹飞。

若i=2，则再输入一个正整数k，表示第j个弹簧的弹力系数被改成k。

输出：对每个i=1的操作，输出一行一个整数表示答案。

数据范围：1≤n≤2×10^5，1≤m≤10^5。

题解：

本题是“单点修改+单点查询”，如果用暴力法，每次查询是O(n)的，m次操作，总复杂度O(mn)，超时。本题的标准解法是动态树LCT，复杂度O(mlogn)。下面用分块求解，编码很简单，

复杂度O(m$\sqrt{n}$)，勉强通过测试。

把整个序列分成$\sqrt{n}$块，对于每个点i，维护两个值：step[i]表示绵羊从第i个点弹出它所在的块所需要的次数、to[i]表示从第i个点所在的块弹出后落到其他块的点。先预处理初始值，复杂度O(n)。

单点查询。从起点出发，根据to[]找到下一个点（这个点在其他块里），累加这个过程中所有的step[]即得到总次数，大于n的时候跳出。最多经过$\sqrt{n}$个块，每块计算一次，复杂度O($\sqrt{n}$)。

单点修改。step[i]和to[i]只与i所在的块有关，与其他块无关，所以单点修改只需要维护一个块，复杂度O($\sqrt{n}$)。

---

1. 《算法竞赛进阶指南》李煜东，225页称为“大段维护，局部朴素”。部分代码参考224页。 ↩

IOI国家集训队论文集1999-2019 09-20
国家集训队论文列表（1999-2019） ___点击目录快速跳转：___ - _国家集训队论文列表（1999-2019）_ * [_…

杜教筛 以及积性函数的前世今生 --算法竞赛专题解析（4） 罗勇军的博客 9711
最全面的杜教筛讲解、最易懂的杜教筛复杂度证明、杜教筛公式、杜教筛模板。

抢沙发
优质评论可以帮助作者获得更高权重
评论

相关推荐

算法竞赛专题解析(20):数论--GCD和LCM_彡倾灬灬染|的博客 5-21
[算法竞赛专题解析(20):数论–GCD和LCM(https://mp.weixin.qq.com/s?__biz=MzIzNzAzNDMzMg==&mid=2456…

算法竞赛专题解析(22):数论--同余_彡倾灬灬染|的博客 4-23
算法竞赛专题解析(22):数论--同余 https://mp.weixin.qq.com/s?__biz=MzIzNzAzNDMzMg==&mid=2456510050…

可持久化线段树（主席树） --算法竞赛专题解析（27） 罗勇军的博客 3987
可持续化线段树（主席树）：来历、经典题、模板

分块算法讲解 XianHaoMing的博客 1万+
分块先简单介绍一下分块算法。 分块算法是一种很常见的根号算法，一般它的时间复杂度会带根号。 分块和线…